

Use-Case Flow

Chat Application WebSocket Programming — Team 5, Section K

Shrujan N | Sameer Deshpande | Soumya A

Use Case ID: UC-05

Use Case Name: Send One-to-One Text Message

Actors: User (Primary, as "Sender")

Description:

This use case describes the step-by-step process for a registered and authenticated user to compose and send a text message to another user (a "Recipient") from their contact list.

Preconditions

1. The Sender must have an active and registered account.
2. The Sender must be successfully logged into the system.
3. The Recipient must be an existing user in the Sender's contact list.
4. The Sender must have an active internet connection.

Postconditions

On Success:

The message is successfully persisted in the database, delivered to the Recipient (if online), and appears in the chat history for both users. The Sender's UI is updated with the sent message and its status.

On Failure:

The message is not sent, and the Sender's UI displays a **"Failed to send"** status with an option to retry. No changes are made to the database or the Recipient's view.

Main Scenario

This flow describes the ideal case where both users are online and everything works perfectly.

Step	Action (Sender)	System Response
1	Sender selects a contact from their contact list.	The system retrieves the chat history with the selected Recipient from the database and displays the chat window.

2	Sender clicks on the text input field.	The system focuses the cursor in the input field. A "typing..." indicator event is sent to the Recipient's client via the WebSocket server.
3	Sender types a message (e.g., "Hello! How are you?").	The system renders the text in the input field as the Sender types.
4	Sender clicks the "Send" button or presses "Enter" .	The system validates that the message content is not empty. The validation passes.
5	—	The system constructs a message object (containing senderID, recipientID, content, timestamp) and sends it to the server over the secure WebSocket (WSS) connection.
6	—	The system immediately renders the new message in the Sender's chat window with a "Sent" status indicator (e.g., a single grey checkmark).

7	—	The server receives the message, persists it to the database, and forwards it to the Recipient's client, who is currently online.
8	—	The server sends an acknowledgement back to the Sender's client that the message has been delivered. The message status updates to "Delivered" (e.g., a double grey checkmark).
—	Use Case Ends	—

Alternate Flow A: Recipient is Offline

This flow begins at **Step 7** of the Main Success Scenario.

Step	Action (Sender)	System Response

7a	—	The server receives the message and persists it to the database. The server attempts to forward the message to the Recipient but finds they are not connected to the WebSocket server.
7b	—	The server flags the message as "Pending" for the offline Recipient. No delivery acknowledgement is sent to the Sender at this time. The message status remains "Sent" (single checkmark).
7c	—	Later, when the Recipient logs in, the system retrieves and delivers all pending messages. At this point, the server sends the delivery acknowledgement to the Sender, updating the status to "Delivered" .
—	Use Case Ends	The message is stored securely until the Recipient comes online.

Exception Flow B: Network Disconnection

This flow begins at **Step 5** of the Main Success Scenario.

Step	Action (Sender)	System Response
5a	—	The system attempts to send the message object to the server but fails due to a lost network connection. The WebSocket connection throws an error.
5b	—	The system catches the error and updates the message in the Sender's UI to a "Failed" status (e.g., a red exclamation mark). An option to "Retry" sending the message is shown.
—	Use Case Ends	The message is held locally on the Sender's client, waiting for user action to retry once the connection is restored.

Exception Flow C: Attempting to Send an Empty Message

This flow begins at **Step 4** of the Main Success Scenario.

Step	Action (Sender)	System Response
4a	—	The system validates the message content and finds it to be empty or containing only whitespace. The validation fails .
4b	—	The system prevents the message from being sent. The "Send" button may be momentarily disabled or the UI provides a subtle shake animation to indicate an invalid action.
—	Use Case Ends	The user is returned to the state at Step 3, with the cursor still in the empty input field.